POSTED 3 YEARS AGO

TorchScript: Tracing vs. Scripting

PyTorch provides two methods to turn an `nn.Module` into a graph represented in TorchScript format: tracing and scripting. This article will:

1. Compare their pros and cons, with a focus on useful tips for tracing.
2. Try to convince you that **`torch.jit.trace` should be preferred over `torch.jit.script`** for deployment of non-trivial models.

The second point might be an uncommon opinion: If I Google "tracing vs scripting", the first article recommends scripting as default. But tracing has many advantages. In fact, by the time I left, **"tracing as default, scripting only when necessary"** is the strategy all detection & segmentation models in Facebook/Meta products are deployed.

Why tracing is better? TL;DR: (i) it will not damage the code quality; (ii) its main limitations can be addressed by mixing with scripting.

Terminology

We start by disambiguate some common terminologies:

- **Export**: refers to the process that turns a model written in eager-mode Python code into a graph that describes the computation.
- **Tracing**: An export method. It runs a model with certain inputs, and "traces / records" all the operations that are executed into a graph.

  `torch.jit.trace` is an export API that uses tracing, used like `torch.jit.trace(model, input)`. See its tutorial and API.
- **Scripting**: Another export method. It parses the Python source code of the model, and compiles the code into a graph.

  `torch.jit.script` is an export API that uses scripting, used like `torch.jit.script(model)`. See its tutorial and API.
- **TorchScript**: This is an **overloaded term**
  - It often refers to the **representation / format** of the exported graph.
  - But sometimes it refers to the scripting **export method**.

  To avoid confusion, I'll never use "TorchScript" alone in this article. I'll use "TS-format" to refer to the format, and "scripting" to refer to the export method.

Because this term is used with ambiguity, it may have caused the impression that "scripting" is the "official / preferred" way to create a TS-format model. But that's not necessarily true.

- **(Torch)Scriptable**: A model is "scriptable" if `torch.jit.script(model)` succeeds, i.e. it exported by scripting.
- **Traceable**: A model is "traceable" if `torch.jit.trace(model, input)` succeeds for a typical input.
- **Generalize**: A traced model (returned object of `trace()`) "generalizes" to other inputs (different from the inputs given during tracing), if it can inference correctly when given other inputs. Scripted models always generalize.
- **Dynamic control flow** or data-dependent control flow: control flow where the operators to be executed depend on the input data, e.g. for a `Tensor` x:
  - `if x[0] == 4: x += 1` is a dynamic control flow.
  - ```
    model: nn.Sequential = ...
    for m in model:
      x = m(x)
    ```

    is NOT a dynamic control flow.

    ```
    class A(nn.Module):
      backbone: nn.Module
      head: Optiona[nn.Module]
      def forward(self, x):
        x = self.backbone(x)
        if self.head is not None:
            x = self.head(x)
        return x
    ```

    is NOT a dynamic control flow.

The Cost of Scriptability

If anyone says "we'll make Python better by writing a compiler for it", you should immediately be alarmed and know that this is extremely difficult. Python is too big and too dynamic. A compiler can only support a subset of its syntax features and builtins, at best -- the scripting compiler in PyTorch is no exception.

What subset of Python does this compiler support? A rough answer is: the compiler has **good support for the most basic syntax, but medium to no support for anything more complicated** (classes, builtins like `range` and `zip`, dynamic types, etc.). But there is no clear answer: even the developers of the compiler usually need to run the code to see if it can be compiled or not.

The incomplete Python compiler limits how users can write code. Though there isn't a clear list of constraints, I can tell from my experience what impact they have had on large projects: **code quality**
    ```

is the cost of scriptability.

Impact on Most Projects

To make their code scriptable / compilable by the scripting compiler, most projects choose to stay on the "safe side" to **only use basic syntax of Python**: no/few custom structures, no builtins, no inheritance, no `Union`, no `**kwargs`, no lambda, no dynamic types, etc.

This is because these "advanced" compiler features are either not supported at all, or with "partial support" which is not robust enough: they may work in some cases but fail in others. And because there is no clear spec of what is supported, users are unable to reason about or workaround the failures. Therefore, eventually users move to and stay on the safe side.

The terrible consequence is that: **developers stop making abstractions / exploring useful language features** due to concerns in scriptability.

A related hack that many projects do is to rewrite part of the code for scripting: create a separate, inference-only forward codepath that makes the compiler happy. This also makes the project harder to maintain.

Impact on Detectron2

Detectron2 supports scripting, but the story was a bit different: it did not go downhill in code quality which we value a lot in research. Instead, with some creativity and direct support from PyTorch team (and some volunteered help from Alibaba engineers), we managed to make most models scriptable without removing any abstractions.

However, it is not an easy task: we had to add dozens of syntax fixes to the compiler, find creative workarounds, and develop some hacky patches in detectron2 that are in this file (which honestly could affect maintainability in the long term). I would not recommend other large projects to aim for "scriptability without losing abstractions" unless they are also closely supported by PyTorch team.

Recommendation

If you think "scripting seems to work for my project" so let's embrace it, I might advise against it for the following reasons, based on my past experiences with a few projects that support scripting:

- **What "works" might be more brittle than you think** (unless you limit yourself to the basic syntax): Your code might happen to compile now, but one day you'll add a few innocent changes to your model and find that the compiler refuses it.
- **Basic syntax is not enough**: Even if more complex abstractions don't appear necessary to your project at the moment, if the project is expected to grow, it will require more language features in the future.

  Take a multi-task detector for example:

  1. There could be 10s of inputs, so it's preferable to use some structures/classes.

2. The same data can have different representations (e.g. different ways to represent a segmentation mask), which demands `Union` or more dynamic types.

3. There are many architectural choices of a detector, which makes inheritance useful.

   Large, growing projects definitely need evolving abstractions to stay healthy.

- **Code quality could severely deteriorate**: Ugly code starts to accumulate, because clean code sometimes just doesn't compile. Also, due to syntax limitations of the compiler, abstractions cannot be easily made to clean up the ugliness. The health of the project gradually goes downhill.

Below is a complaint in PyTorch issues. The issue itself is just one small papercut of scripting, but similar complaints were heard many times. The status-quo is: scripting forces you to write ugly code, so only use it when necessary.



Make a Model Trace and Generalize

The Cost of Traceability

What it takes to make a model traceable is very clear, and has a much smaller impact on code health.

1. First, neither scripting nor tracing works if the model is not even a proper single-device, connected graph representable in TS-format. For example, if the model has `DataParallel` submodules, or if the model converts tensors to numpy arrays and calls OpenCV functions, etc, you'll have to refactor it.

   Apart from this obvious constraint, there are only two extra requirements for traceability.

2. **Input/output format**

   Model's inputs/outputs have to be `Union[Tensor, Tuple[Tensor], Dict[str, Tensor]]` or their nested combinations. Note that values in a dict have to belong to the same type.

   Similar constraints exist for scripting as well. However, in tracing the constraint does not apply to submodules: submodules can use **any** input/output format: dicts of Any, classes, kwargs, anything that Python supports. Only the top-level model is required to use the constraint format.

   This makes the constraint very easy to satisfy. If the model uses richer formats, just create a simple wrapper around it that converts to/from `Tuple[Tensor]`. Detectron2 even automates this for all its models by a universal wrapper like this:

```
outputs = model(inputs)   # inputs/outputs are rich structure, e.g. dicts or classe
# torch.jit.trace(model, inputs)  # FAIL! unsupported format
adapter = TracingAdapter(model, inputs)
```

```
traced = torch.jit.trace(adapter, adapter.flattened_inputs)  # Can now trace the mo

# Traced model can only produce flattened outputs (tuple of tensors):
flattened_outputs = traced(*adapter.flattened_inputs)
# Adapter knows how to convert it back to the rich structure (new_outputs == output
new_outputs = adapter.outputs_schema(flattened_outputs)
```

Automatically Flatten & Unflatten Nested Containers has more details on how this adapter is implemented.

3. **Symbolic shapes**:

   Expressions like `tensor.size(0)`, `tensor.size()[1]`, `tensor.shape[2]` are integers in eager mode, but `Tensor`s in tracing mode. Such difference is necessary so that during tracing, shape computation can be captured as symbolic operations in the graph. An example is given in the next section about generalization.

   Due to different return types, a model may be untraceable if parts of it assume shapes are integers. This usually can be fixed quite easily by handling both types in the code. A helpful function is `torch.jit.is_tracing` which checks if the code is executed in tracing mode.

That's all it takes for traceability - most importantly, **any Python syntax** is allowed in model implementation, because tracing does not care about syntax at all.

Generalization Problem

Just being "traceable" is not sufficient. The biggest problem with tracing, is that it may not generalize to other inputs. This problem happens in the following cases:

1. **Dynamic control flow**:

```
>>> def f(x):
...   return torch.sqrt(x) if x.sum() > 0 else torch.square(x)
>>> m = torch.jit.trace(f, torch.tensor(3))
>>> print(m.code)
def f(x: Tensor) -> Tensor:
  return torch.sqrt(x)
```

   In this example, due to dynamic control flow, the trace only keeps one branch of the condition, and will not generalize to certain (negative) inputs.

2. **Capture variables as constants**:

```
>>> a, b = torch.rand(1), torch.rand(2)
>>> def f1(x): return torch.arange(x.shape[0])
>>> def f2(x): return torch.arange(len(x))
>>> # See if the two traces generalize from a to b:
>>> torch.jit.trace(f1, a)(b)
tensor([0, 1])
>>> torch.jit.trace(f2, a)(b)
```

```
>>> torch.jit.trace(f2, a)(b)
tensor([0])  # WRONG!
>>> # Why f2 does not generalize? Let's compare their code:
>>> print(torch.jit.trace(f1, a).code, torch.jit.trace(f2, a).code)
def f1(x: Tensor) -> Tensor:
  _0 = ops.prim.NumToTensor(torch.size(x, 0))
  _1 = torch.arange(annotate(number, _0), dtype=None, layout=0, device=torch.device
  return _1
def f2(x: Tensor) -> Tensor:
  _0 = torch.arange(1, dtype=None, layout=0, device=torch.device("cpu"), pin_memory
  return _0
```

Intermediate computation results of a non-Tensor type (in this case, an int type) may be captured as constants, using the value observed during tracing. This causes the trace to not generalize.

In addition to `len()`, this issue can also appear in:

- `.item()` which converts tensors to int/float.
- Any other code that converts torch types to numpy/python primitives.
- A few problematic operators, e.g. advanced indexing.

3. **Capture device**:

```
>>> def f(x):
...   return torch.arange(x.shape[0], device=x.device)
>>> m = torch.jit.trace(f, torch.tensor([3]))
>>> print(m.code)
def f(x: Tensor) -> Tensor:
  _0 = ops.prim.NumToTensor(torch.size(x, 0))
  _1 = torch.arange(annotate(number, _0), dtype=None, layout=0, device=torch.device
  return _1
>>> m(torch.tensor([3]).cuda()).device
device(type='cpu')  # WRONG!
```

Similarly, operators that accept a `device` argument will remember the device used during tracing (this can be seen in `m.code`). So the trace may not generalize to inputs on a different device. Such generalization is almost never needed, because deployment usually has a target device.

Let Tracing Generalize

The above problems are annoying and often silent (warnings, but no errors), but they can be successfully addressed by good practice and tools:

- **Pay attention to `TracerWarning`**: In the first two examples above, `torch.jit.trace` actually emits warnings. The first example prints:

```
a.py:3: TracerWarning: Converting a tensor to a Python boolean might cause the trac
```

```
We can't record the data flow of Python values, so this value will be treated as a
This means that the trace might not generalize to other inputs!
if x.sum() > 0:
```

Paying attention to these warnings (or even better, catch them) will expose most general problems of tracing.

Note that the "capture device" case does not print warnings because tracing was not designed to support such generalization at all.

- **Unittests for parity**: Unittests should be done after export and before deployment, to verify that the exported model produces the same outputs as the original eager-mode model, i.e.

```
assert allclose(torch.jit.trace(model, input1)(input2), model(input2))
```

If generalization across shapes is needed (not always needed), `input2` should have different shapes from `input1`.

Detectron2 has many generalization tests, e.g. this and this. Once a gap is found, inspecting the code of the exported TS-format model can uncover the place where it fails to generalize.

- **Avoid unnecessary "special case" conditions**: Avoid conditions like

```
if x.numel() > 0:
  output = self.layers(x)
else:
  output = torch.zeros((0, C, H, W))  # Create empty outputs
```

that handles special cases such as empty inputs. Instead, improve `self.layers` or its underlying kernel so it supports empty inputs. This would result in cleaner code and also improve tracing. This is why I'm involved in many PyTorch issues that improve support for empty inputs, such as #12013, #36530, #56998. Most PyTorch operations work perfectly with empty inputs, so such branching is hardly needed.

- **Use symbolic shapes**: As mentioned earlier, `tensor.size()` returns `Tensor` during tracing, so that shape computations are captured in the graph. Users should avoid accidentally turning tensor shapes into constants:
  - Use `tensor.size(0)` instead of `len(tensor)` because the latter is an int. For custom classes, implement a `.size` method or use `.__len__()` instead of `len()`, e.g. like here.
  - Do not convert sizes by `int()` or `torch.as_tensor` because they will capture constants. This helper function is useful to convert sizes into a tensor, in a way that works in both tracing and eager mode.
- **Mix tracing and scripting**: they can be mixed together, so you can use scripting on the small portion of code that tracing does not work correctly. This can **fix almost all problems of tracing**. More on this below.

Mix Tracing and Scripting

Tracing and scripting both have their own problems, and the best solution is usually to mix them together. This gives us **the best of both worlds**.

To minimize the negative impact on code quality, we should use tracing for the majority of log
use scripting only when necessary.

1. **Use** `@script_if_tracing`: Inside `torch.jit.trace`, the `@script_if_tracing` decorator can compile functions by scripting. Typically, this only requires a small refactor of the forward logic to separate the parts that need to be compiled (the parts with control flow):

   ```
   def forward(self, ...):
     # ... some forward logic
     @torch.jit.script_if_tracing
     def _inner_impl(x, y, z, flag: bool):
         # use control flow, etc.
         return ...
     output = _inner_impl(x, y, z, flag)
     # ... other forward logic
   ```

   By scripting only the parts that need it, the code quality damage is strictly smaller than making the entire model scriptable, and it does not affect the module's forward interface at all.

   The function decorated by `@script_if_tracing` has to be a pure function that does not contain modules. Therefore, sometimes a bit more refactoring is needed:

   Before Refactoring

   ```
   # This branch cannot be compiled by
   # @script_if_tracing, because it
   # refers to `self.layers`
   if x.numel() > 0:
     x = preprocess(x)
     output = self.layers(x)
   else:
     # Create empty outputs
     output = torch.zeros(...)
   ```

   After Refactoring

   ```
   # This branch can be compiled by @script_if_t
   if x.numel() > 0:
     x = preprocess(x)
   else:
     # Create empty inputs
     x = torch.zeros(...)
   # Needs to make sure self.layers accept empty
   # inputs. If necessary, add such condition br
   # into self.layers as well.
   output = self.layers(x)
   ```

   In fact, for most vision models, dynamic control flow is needed only in a few submodules where it's easy to be scriptable. To show how rare it is needed, the entire detectron2 only has two functions decorated with `@script_if_tracing` due to control flows: paste_masks and heatmaps_to_keypoints, both for post-processing only. A few other functions are also decorated to generalize across devices (a very rare requirement).

2. **Use scripted / traced submodules**:

   ```
   model.submodule = torch.jit.script(model.submodule)
   torch.jit.trace(model, inputs)
   ```

   In this example, suppose `submodule` cannot be traced correctly, we can script it before tracing

In this example, suppose `submodule` cannot be traced correctly, we can script it before tracing. However I do not recommend it. If possible, I will suggest using `@script_if_tracing` inside `submodule.forward` instead, so that scripting is limited to the internals of the submodule without affecting the module's interface.

And similarly,

```
model.submodule = torch.jit.trace(model.submodule, submodule_inputs)
torch.jit.script(model)
```

this uses a traced submodule during scripting. This looks nice, but is not so useful in practice: it will affect the interface of `submodule`, requiring it to only accept/return `Tuple[Tensor]` -- this is a big constraint that might hurt code quality even more than scripting.

A rare scenario where "tracing a submodule" is useful, is this:

```
class A(nn.Module):
  def forward(self, x):
    # Dispatch to different submodules based on a dynamic, data-dependent condition
    return self.submodule1(x) if x.sum() > 0 else self.submodule2(x)
```

`@script_if_tracing` cannot compile such control flow because it only supports pure functions. If `submodule{1,2}` are complex and cannot be scripted, using traced submodules in a scripted parent `A` is the best option.

3. **Merge multiple traces**:

   Scripted models support two more features that traced models don't:

   - **Control flow conditioned on attributes**: a scripted module can have mutable attributes (e.g. a boolean flag) that affect control flows. Traced modules do not have control flows.
   - **Multiple methods**: a traced module only supports `forward()`, but a scripted module can have multiple methods.

   Actually, both features above are doing the same thing: they allow an exported model to be used in different ways, i.e. execute different sequences of operators as requested by the caller.

   Below is an example scenario where such feature is useful: if `Detector` is scripted, the caller can mutate its `do_keypoint` attribute to control its behavior, or call `predict_keypoint` method directly if needed.

```
class Detector(nn.Module):
  do_keypoint: bool

  def forward(self, img):
      box = self.predict_boxes(img)
      if self.do_keypoint:
```

```
            kpts = self.predict_keypoint(img, box)

    @torch.jit.export
    def predict_boxes(self, img): pass

    @torch.jit.export
    def predict_keypoint(self, img, box): pass
```

This requirement is not seen very often. But if needed, how to achieve this in tracing? I have a solution that's not very clean:

Tracing can only capture one sequence of operators, so the natural way is to trace the model twice:

```
det1 = torch.jit.trace(Detector(do_keypoint=True), inputs)
det2 = torch.jit.trace(Detector(do_keypoint=False), inputs)
```

We can then alias their weights (to not duplicate the storage), and merge the two traces into one module to script.

```
det2.submodule.weight = det1.submodule.weight
class Wrapper(nn.ModuleList):
  def forward(self, img, do_keypoint: bool):
    if do_keypoint:
        return self[0](img)
    else:
        return self[1](img)
exported = torch.jit.script(Wrapper([det1, det2]))
```

Performance

If a model is both traceable and scriptable, **tracing always generates same or simpler graph** (therefore likely faster).

Why? Because scripting tries to faithfully represent your Python code, even some of it are unnecessary. For example: it is not always smart enough to realize that some loops or data structures in the Python code are actually static and can be removed:

```
class A(nn.Module):
  def forward(self, x1, x2, x3):
    z = [0, 1, 2]
    xs = [x1, x2, x3]
    for k in z: x1 += xs[k]
    return x1
model = A()
print(torch.jit.script(model).code)
# def forward(self, x1: Tensor, x2: Tensor, x3: Tensor) -> Tensor:
#   z = [0, 1, 2]
```

```
#   xs = [x1, x2, x3]
#   x10 = x1
#   for _0 in range(torch.len(z)):
#     k = z[_0]
#     x10 = torch.add_(x10, xs[k])
#   return x10
print(torch.jit.trace(model, [torch.tensor(1)] * 3).code)
# def forward(self, x1: Tensor, x2: Tensor, x3: Tensor) -> Tensor:
#   x10 = torch.add_(x1, x1)
#   x11 = torch.add_(x10, x2)
#   return torch.add_(x11, x3)
```

This example is very simple, so it actually has workarounds for scripting (use tuple instead of list), or the loop might get optimized in a later optimization pass. But the point is: the graph compiler is not always smart enough. For complicated models, scripting might generate a graph with unnecessary complexity that's hard to optimize.

Concluding Thoughts

Tracing has clear limitations: I spent most of this article talking about the limitations of tracing and how to fix them. I actually think this is the advantage of tracing: it has **clear** limitations (and solutions), so you can reason about whether it works.

On the contrary, scripting is more like a black box: no one knows if it works before trying. I didn't mention a single trick about how to fix scripting: there are many of them, but it's not worth your time to probe and fix a black box.

Tracing has small blast radius: Both tracing and scripting affect how code can be written, but tracing has a much smaller blast radius, and causes much less damage:

- It limits the input/output format, but on the outer-most module only. (And this issue can be automatically solved as discussed above.)
- It needs some code changes to generalize (e.g. to mix scripting in tracing), but these changes only go into the internal implementation of the affected modules, not their interfaces.

On the other hand, scripting has an impact on:

- The interface of every module & submodule involved.
  - IMO, this is the biggest damage: Advanced syntax features are needed in interfaces, and I'm not willing to compromise on interface design.
  - This may end up affecting training as well because interface is often shared between training and inference.
- Pretty much every line of code in the inference forward path.

Having a large blast radius is why scripting can do great harm to code quality.

Control flow vs. other Python syntax: PyTorch is loved by its users because they can "just write

Python", and most importantly write Python control flows. But other syntax of Python are important as well. If being able to write Python control flow (scripting) means losing other great syntax, I'd rather give up on the ability to write Python control flow.

In fact, if PyTorch is less obsessed with Python control flow, and offers me symbolic control fl such as `torch.cond` like this (similar to the API of `tf.cond`):

```
def f(x):
  return torch.cond(x.sum() > 0, lambda: torch.sqrt(x), lambda: torch.square(x))
```

Then `f` could be traced correctly and I would be happy to use this, no longer having to worry about scripting. TensorFlow AutoGraph is a great example that automates this idea.

Deep Learning PyTorch

‹ Automatically Flatten & Unflatten Nested Containers
谈谈Github上如何交流(4): {Feature,Pull} Request ›

Comments

9 Comments 1 Login ▼

Join the discussion...

LOG IN WITH OR SIGN UP WITH DISQUS

Name

6 • Share Best Newest Oldest

David Cheng
7 months ago
What a wonderful blog post! I wish I had read this earlier. I am currently working on a frontend converting tool for our compiler. It took a considerable amount of effort to make the llama model with a loop, generated by torch scripting, compatible with our compiler.

1 0 Reply

Cyrill
a year ago

This is an extremly helpful blog. This is what the official documentation are lacking. Thank you so much.

1 0 Reply

兰铃

2 years ago

Best blog about torchscript I've seen so far

1 0 Reply

xunzhang

2 years ago

Hi, does the `TracingAdapter ` works for string input? Wanna tracing input as string type(or list of str) as input type, how to do that?

0 0 Reply

xunzhang xunzhang

2 years ago

figured out a way. use unicode encoding to numeric of each char in string and in forward function, decode first and do tokenize, ... etc.

0 0 Reply

Akshit Arora

2 years ago

Thanks for this post, incredibly useful!

0 0 Reply

Fabian

2 years ago

Thanks for the thorough and illuminating post.

Did the requirements for traceability change?

You mention:

> Model's inputs/outputs have to be Union[Tensor, Tuple[Tensor]] to be traceable.

However, the docs say

> Tracing is ideal for code that operates only on Tensors and lists, dictionaries, and tuples of Tensors.

Futhermore, the following codes seems to run for me:

```
import typing as t
import torch


def foo(inputs: t.Dict[str, torch.Tensor]) -> torch.Tensor:
```

see more

0 0 Reply



Yuxin Wu Mod → Fabian

2 years ago

Yeah that works, I should update the post. But it still requires all values in the dict to have the same type, e.g. this doesn't work `inp = {"x":torch.rand(3), "y":{"a": torch.rand(3)}}`. So it would still require a "flattening" step for slightly complicated inputs.

1 0 Reply

jc

2 years ago

Extremely helpful for understanding the differences in tracing vs scripting and when they should be used

0 0 Reply

Subscribe Privacy Do Not Sell My Data DISQUS



Yuxin Wu

© 2025 Yuxin Wu

License: